DEPARTMENT OF INDUSTRIAL MANAGEMENT

FACULTY OF SCIENCE | UNIVERSITY OF KELANIYA

INTE 22283 – Mobile Application Development

Week 4: Mobile Application Architecture

Sammani Maddumahewage

Lecturer (Probationary)

BSc (Hons) in Information Technology

Today's Learning Outcomes

- Explain the role of software architecture in mobile application development
- Describe the MVC (Model-View-Controller) architectural pattern
- Describe the MVVM (Model-View-ViewModel) architectural pattern
- Explain Android application components: Activities, Services, Broadcast Receivers, Content Providers
- Describe the Activity and Fragment lifecycle
- Discuss state management and data binding in mobile applications
- Apply separation of concerns when structuring a mobile project
- Implement a simple multi-screen application using an architectural pattern

Recap and Framing

- **Previous weeks:** We chose a platform approach and built our first running app
- **This week:** We address how to structure the code inside an app so it stays maintainable as it grows beyond a single screen
- **Why this matters now:** Without a clear architecture, even a small app's code quickly becomes tangled and difficult to test or extend

Why Software Architecture Matters

- **Definition:** Software architecture is the high-level structure of a system — how its components are organized and how they interact
- **Without architecture:** UI code, business logic, and data access become mixed together, making the app hard to test, debug, and extend
- **With clear architecture:** Each part of the app has a well-defined responsibility, so changes in one area don't unexpectedly break another
- **Mobile-specific pressure:** Mobile apps must also gracefully handle lifecycle events (rotation, backgrounding) — architecture patterns provide a disciplined way to do this consistently

MVC: Model-View-Controller

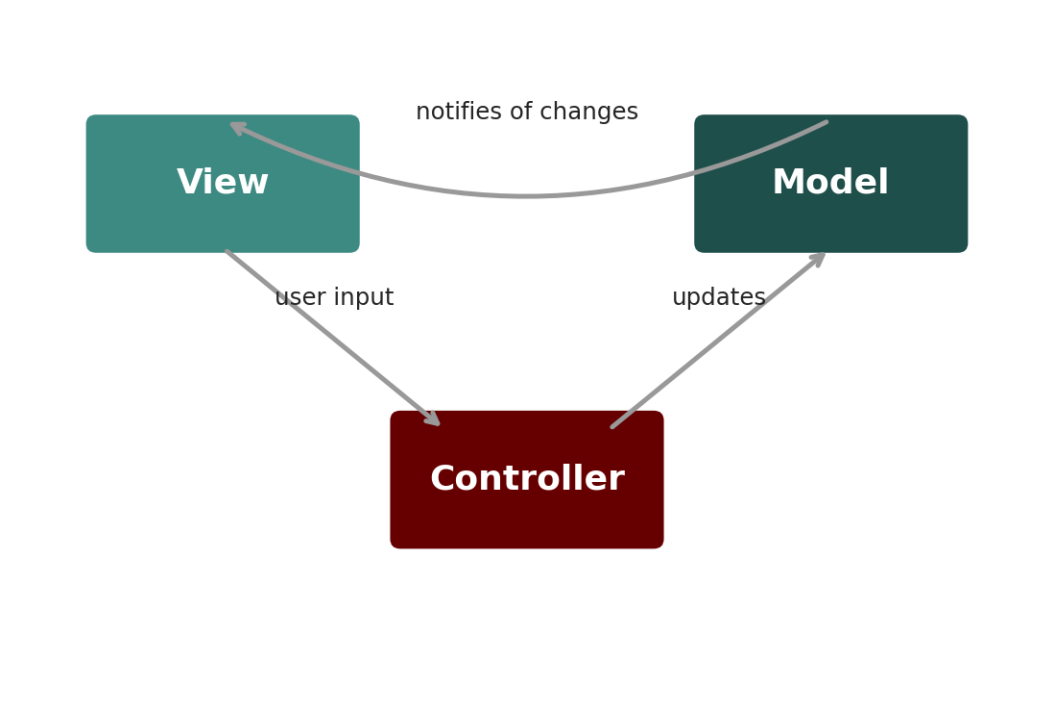


Figure 1: The Controller mediates between the Model (data) and the View (UI)

- **Model:** Holds the application's data and business logic, independent of the UI
- **View:** Displays data to the user and captures user input
- **Controller:** Receives input from the View, updates the Model, and refreshes the View

MVC in Android Practice

- **Typical mapping:** The Activity or Fragment often plays the role of both View and Controller, while separate classes represent the Model
- **Strength:** Simple to understand and quick to implement for small apps
- **Common weakness:** Activities tend to grow into “massive” classes handling both UI logic and control logic, making testing difficult
- **Why this matters:** This weakness is the main motivation behind MVVM, which we examine next

MVVM: Model-View-ViewModel

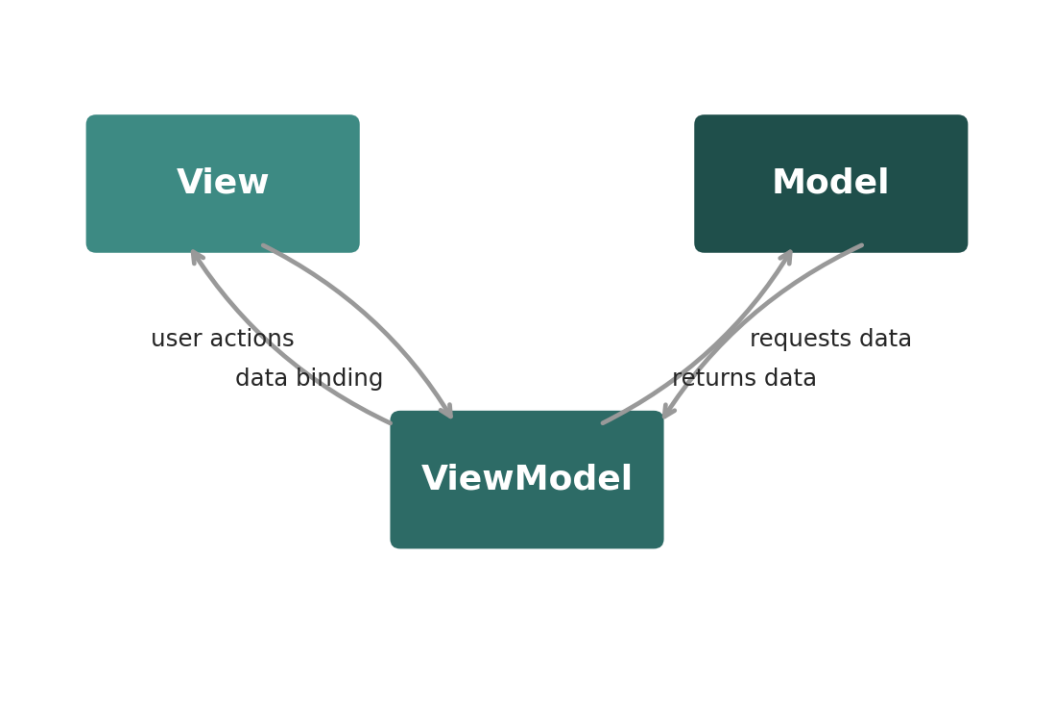


Figure 2: The ViewModel exposes data the View observes, without holding a direct reference to the View

- **Model:** Same role as in MVC — application data and business logic
- **View:** Observes the ViewModel and updates automatically when its data changes
- **ViewModel:** Holds UI-related state and logic, exposing observable data without referencing the View directly

Why MVVM Improves on MVC for Mobile Apps

- **Lifecycle survival:** Android's ViewModel class is specifically designed to survive configuration changes like screen rotation, solving a common MVC pain point
- **Testability:** Because the ViewModel doesn't directly reference UI elements, its logic can be unit-tested without running the full UI
- **Data binding:** The View can automatically reflect ViewModel changes through observable data structures (e.g., LiveData or StateFlow), reducing manual update code
- **Trade-off:** MVVM introduces more initial structure and concepts to learn than MVC, which can feel like overhead for very simple apps

MVC vs. MVVM at a Glance

Table 1: Comparing the two patterns across key dimensions

Aspect	MVC	MVVM
View-logic coupling	Tighter (Activity often does both)	Looser (ViewModel independent of View)
Testability	Harder	Easier
Survives rotation	Not by default	Yes, with ViewModel
Learning curve	Lower	Slightly higher
Best for	Small, simple apps	Apps expected to grow and scale

The Four Core Android Components

- **Activities**
  - Represent a single screen with a user interface; the building block of user-facing app navigation
- **Services**
  - Run operations in the background without a UI, such as playing music or syncing data
- **Broadcast Receivers**
  - Listen for and respond to system-wide or app-wide announcements (e.g., battery low, network changed)
- **Content Providers**
  - Manage shared access to structured data, enabling controlled data sharing between apps

How the Four Components Work Together

- **Example scenario:** A messaging app receives a new message while running in the background
- **Broadcast Receiver:** Detects the incoming message notification from the system
- **Service:** Downloads the message content in the background, even if the app isn't currently open
- **Content Provider:** Stores the message in a shared database accessible to other parts of the app (or other apps, if permitted)
- **Activity:** Displays the message to the user once they open the app

Recap: The Activity Lifecycle

- **From Week 2:** Activities move through Created → Started → Resumed → Paused → Stopped → Destroyed states
- **Why we revisit it here:** Architecture patterns like MVVM exist specifically to make managing this lifecycle easier and less error-prone
- **Key architectural insight:** Any data the user shouldn't lose must live somewhere that survives the Activity's destruction — which is exactly what a ViewModel provides

The Fragment Lifecycle

- **What is a Fragment?**
  - A reusable portion of UI that lives inside an Activity, enabling modular, multi-pane, or tabbed interfaces
- **Fragment lifecycle is layered on the Activity's**
  - Fragments have their own onCreateView(), onViewCreated(), onDestroyView() callbacks in addition to Activity-style callbacks
- **Why Fragments add complexity**
  - A Fragment's view can be destroyed and recreated independently of the Fragment object itself, a common source of subtle bugs
- **Practical implication**
  - Using a ViewModel scoped to the Fragment (or shared with its parent Activity) avoids holding stale view references

State Management in Mobile Apps

- **Definition:** The practice of tracking and updating the data that determines what the UI currently displays
- **UI state:** Information directly tied to what's on screen right now (e.g., whether a loading spinner is visible)
- **Persistent state:** Data that must survive beyond the current screen session (e.g., user login status, saved drafts)
- **Why it's harder on mobile:** Frequent lifecycle events (rotation, backgrounding, low-memory termination) mean state can be lost far more easily than in a desktop application

Data Binding

- **Definition:** A technique that automatically synchronizes UI elements with underlying data, removing the need to manually update each view
- **Without data binding:** Developers must write explicit code every time data changes to update each affected UI element
- **With data binding:** The View observes data (e.g., from a ViewModel) and updates itself automatically when that data changes
- **Connection to MVVM:** Data binding is what makes the View-ViewModel relationship in MVVM practical and low-boilerplate

Separation of Concerns in Practice

- **Principle:** Each part of a system should have one clear responsibility, and should not need to know the implementation details of other parts
- **UI layer:** Displays data and forwards user actions; contains no business logic
- **Logic layer (ViewModel):** Coordinates what data should be shown and how user actions should be handled
- **Data layer (Model / Repository):** Handles where data actually comes from — local database, network, or cache — hidden behind a simple interface
- **Benefit:** Each layer can be changed, tested, or replaced independently — for example, swapping a local database for a cloud one without touching the UI

Worked Example: A To-Do List App

- **Without architecture:** The Activity directly creates, stores, and displays to-do items all in one class — a single 500-line file
- **Applying MVVM:** A TodoViewModel holds the current list and exposes it as observable data; a TodoRepository (Model) manages storage; the Activity/Fragment (View) just displays what the ViewModel provides
- **Result of separation:** The storage mechanism could change from an in-memory list to a database without changing a single line of UI code
- **Testing benefit:** The ViewModel's logic (e.g., marking a task complete) can be unit-tested without launching the app's UI at all

Building a Multi-Screen App Using MVVM

- **Step 1:** Define the Model — a simple data class representing the information your app manages
- **Step 2:** Create a ViewModel that holds and exposes that data, with functions to update it
- **Step 3:** Build two Activities or Fragments (Views) that observe the same ViewModel's data
- **Step 4:** Use navigation (e.g., Intents or the Navigation component) to move between screens
- **Result:** Both screens stay in sync automatically because they observe the same underlying ViewModel state

CONCLUSION

Key Takeaways

- Software architecture organizes an app's code so it stays maintainable, testable, and easy to extend as it grows
- MVC separates Model, View, and Controller, but Android Activities often end up handling both View and Controller responsibilities
- MVVM improves on MVC by introducing a ViewModel that survives configuration changes and can be tested independently of the UI
- Android's four core components — Activities, Services, Broadcast Receivers, Content Providers — each have a distinct, well-defined role
- Separation of concerns, state management, and data binding together make multi-screen apps easier to build and reason about

Summary

- This week we moved from building a single working app to structuring apps that can grow. We compared MVC and MVVM, examined Android's four core application components, and revisited the Activity/Fragment lifecycle through an architectural lens.

- We connected state management and data binding directly to why MVVM has become the dominant pattern in modern Android development, and applied separation of concerns to a worked multi-screen example.

- Next week, we shift focus from internal structure to what users actually see: mobile UI design principles.

Discussion Questions

- Why might a team choose MVC for a small internal tool but MVVM for a flagship consumer app? What factors drive that decision?
- Which Android component (Activity, Service, Broadcast Receiver, Content Provider) would you use to play music in the background while the user browses other apps, and why?
- How does the ViewModel's ability to survive configuration changes solve a problem you saw in Week 2's lifecycle discussion?
- Can you think of a scenario where strict separation of concerns might be excessive for a very small project? What signals would tell you when added structure is worth the cost?

Thank You !

Questions?

Email: Sammani@kln.ac.lk | Department of Industrial Management